

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job
This AWS Glue job processes customer and order data through a complete ETL pipeline with data quality controls and Slowly Changing Dimension (SCD) Type 2 tracking.---

High-Level Flow

1. **Initialize** Spark/Glue contexts and load configuration
2. **Ingest** customer and order CSV files from S3
3. **Clean** data by removing nulls and duplicates
4. **Register** cleaned tables in Glue Data Catalog
5. **Generate** order summary with customer details
6. **Aggregate** customer spending metrics
7. **Apply** SCD Type 2 columns for historical tracking---

Step-by-Step Breakdown

****Step 1: Configuration Loading****

```
get_job_parameters()
```

- Attempts to load configuration from `config/glue\_params.yaml` - Falls back to hardcoded defaults if file not found
- Returns dictionary with S3 paths, table names, format options, and feature flags

****Step 2: Context Initialization****

```
initialize_spark_contexts()
```

- Detects if running in test vs production environment
- Creates SparkContext, GlueContext, SparkSession, and Job objects
- Returns all four contexts for use throughout the job

****Step 3: Helper Classes Setup****

Two utility classes encapsulate I/O operations:
****DataReader****- `read_csv()`: Reads CSV files with configurable schema, headers, delimiters
****DataWriter****- `write_dataframe()`: Dispatches to format-specific writers (Parquet, CSV, JSON)
- Internal methods handle actual write operations with mode control

****Step 4: Customer Data Ingestion** (TR-INGEST-001)**

```
ingest_customer_data(reader, params)
```

1. Validates S3 path accessibility
2. Defines schema: CustId, Name, EmailId, Region (all strings)
3. Reads CSV from `customer\_source\_path`
4. Normalizes column names to lowercase
5. Returns DataFrame with customer records

****Step 5: Order Data Ingestion** (TR-INGEST-002)**

```
ingest_order_data(reader, params)
```

1. Validates S3 path accessibility
2. Defines schema: OrderId, ItemName, PricePerUnit (decimal), Qty (int), Date, CustId
3. Reads CSV from `order\_source\_path`
4. Converts Date column from string to DateType
5. Returns DataFrame with order records

****Step 6: Customer Data Cleaning** (TR-CLEAN-001)**

```
clean_customer_data(customer_df, params)
```

1. Removes rows with actual NULL values using ``na.drop()``
2. Removes rows containing string representations of null ('Null', 'NULL', 'null')
3. Removes duplicate rows if ``remove_duplicates`` flag is enabled
4. Returns cleaned DataFrame

****Step 7: Order Data Cleaning** (TR-CLEAN-002)**

```
clean_order_data(order_df, params)
```

1. Removes rows with actual NULL values
2. Checks only string columns (orderid, itemname, custid) for string null values
3. Removes duplicates if enabled
4. Returns cleaned DataFrame

****Step 8: Write Cleaned Data****

```
write_data_safe(writer, df, path, format)
```

- Writes cleaned customer data to ``customer_clean_target_path``
- Writes cleaned order data to ``order_clean_target_path``
- Uses Parquet format by default (configurable)

****Step 9: Catalog Registration** (TR-CATALOG-001, TR-CATALOG-002)**

```
register_catalog_table(writer, df, params, table_type)
```

- For both customer and order tables:
1. Writes DataFrame to catalog-specific S3 location
 2. Makes data queryable through Glue Data Catalog
 3. Enables downstream analytics and queries

****Step 10: SCD Type 2 Column Addition****

```
add_scd2_columns(df)
```

- Adds four columns for historical tracking:- ``isactive``: Boolean flag (True for current records)- ``startdate``: Timestamp when record became active- ``enddate``: Timestamp when record became inactive (NULL for active)- ``opts``: Operational timestamp

****Step 11: Order Summary Generation****

```
generate_order_summary_scd2(customer_clean, order_clean, writer, params)
```

1. Performs inner join between orders and customers on ``custid``
2. Selects order details plus customer name, email, region
3. Adds SCD Type 2 columns if enabled
4. Writes to ``order_summary_target_path``
5. Creates denormalized view combining both datasets

****Step 12: Customer Aggregate Spend** (TR-AGG-001)**

```
generate_customer_aggregate_spend(customer_clean, order_clean, writer, params)
```

1. Calculates ``totalamount`` = ``priceperunit`` × ``qty`` for each order
2. Joins orders with customer data
3. Groups by customer (custid, name, email, region) and date
4. Aggregates: - ``totalspend``: Sum of all order amounts - ``ordercount``: Count of orders
5. Adds SCD Type 2 columns
6. Writes to ``customer_aggregate_spend_path``

****Step 13: Main Execution****

```
main()
```

Orchestrates the entire pipeline:1. Initializes contexts and loads parameters2. Executes all ingestion, cleaning, and transformation steps in sequence3. Handles errors with try-except-finally block4. Commits the Glue job on completion5. Ensures job state is properly finalized even on failure---

Key Design Patterns

Error Handling: All I/O operations wrapped in safe functions with exception handling
Schema Enforcement: Explicit schemas defined for both datasets prevent type inference issues
Configurability: YAML-based configuration with fallback defaults enables environment flexibility
Modularity: Each transformation step isolated in dedicated function with clear responsibility
SCD Type 2: Enables historical tracking of dimension changes over time
Data Quality: Multi-stage null removal (actual NULLs + string representations) plus deduplication---

Output Artifacts

The job produces five datasets:1. **customer_clean**: Cleaned customer dimension2. **order_clean**: Cleaned order facts3. **catalog tables**: Queryable versions in Glue Data Catalog4. **order_summary**: Denormalized orders with customer details5. **customer_aggregate_spend**: Daily customer spending metricsAll outputs include SCD Type 2 columns for temporal analysis.